

이승주

Backend Developer



Java와 Spring Boot로 백엔드를 개발하는 이승주입니다. 기계공학을 공부한 뒤 SSAFY에서 개발을 시작했고, 현재 SW마에스트로에서 Lorekeeper를 만들고 있습니다.

로그인 이후 요청이 어디서 막히는지, 데이터와 권한의 경계를 어디에 둘지 고민하며 개발합니다. ILOG에서는 인증과 Gateway를, Matching SSAFY에서는 팀과 실시간 채팅을 담당했습니다.

프로젝트

01 · ILOG →

2025.10 — 2025.11

인증 · 사용자 · 그룹 · API Gateway

02 · Lorekeeper →

2026.05 — 진행 중

로그인 · 작품 · 회차

03 · Matching SSAFY →

2025.07 — 2025.08

팀 · 실시간 채팅

교육 · 활동

2026.05 — 2026.11 예정

SW마에스트로 17기

연수생 · 참여 중

Lorekeeper 로그인 · Work · Episode 개발

2025.01 — 12

삼성 청년 SW·AI 아카데미 13기

Java 백엔드 개발과 팀 프로젝트

개발 환경과 작업 방식

개인 Linux 홈서버를 학습 · 개발에 활용합니다. AI 에이전트의 작업 분해와 완료 기준을 설계하고, 홈서버의 Terraform 전환 작업에 적용했습니다.

 github.com/ju1115

인증·사용자·그룹·API Gateway

6인 팀 · 네 서비스 단독 설계·구현

[GitHub 저장소](#) [상세 웹페이지](#) [기](#)

프로젝트 개요

일기와 일상을 기록하고 감정 분석 결과를 확인하는 서비스입니다. 인증·사용자·그룹·API Gateway 네 서비스를 단독 설계하고 구현했습니다.

주요 기능

- 일기 작성과 감정 분석 결과 확인
- 그룹을 통한 기록 공유
- Google 소셜 로그인

담당 범위

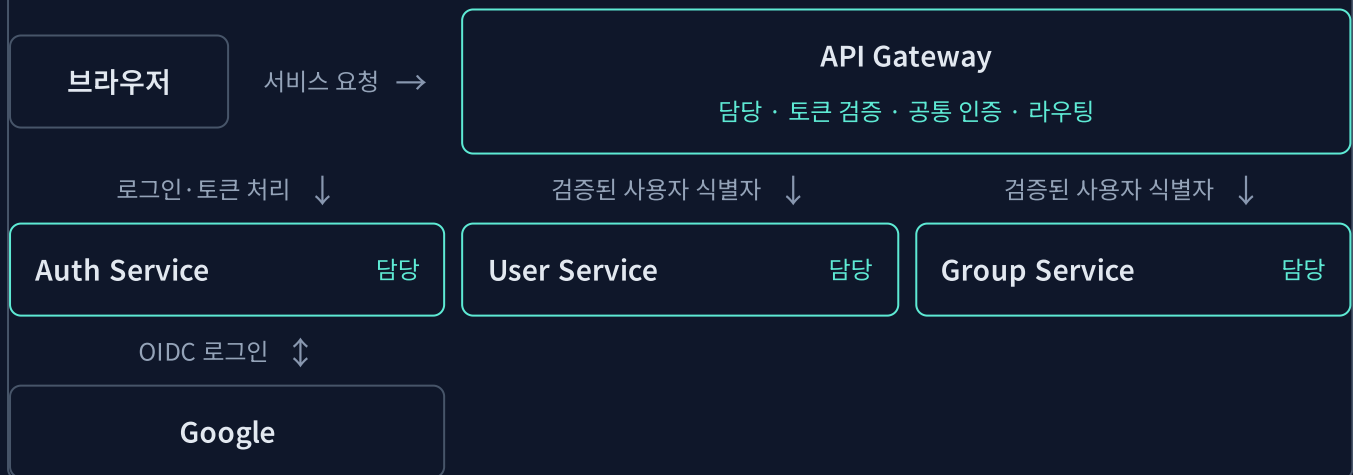
- 로그인과 토큰 발급·재발급, 로그아웃
- 내 정보 조회와 사용자 프로필 관리
- 그룹 생성·조회, 초대 코드와 그룹 참여
- 서비스 라우팅과 공통 인증 처리

[Java](#)[Spring Security](#)[Gateway](#)

아키텍처

인증과 내부 서비스 요청 흐름

담당 · 청록색 테두리와 역할 레이블



API Gateway가 서비스별 요청을 전달하고 쿠키의 액세스 토큰을 공개 키로 검증합니다. Auth Service는 Google 로그인과 토큰 발급을 맡습니다. User·Group Service는 Gateway가 전달한 사용자 식별자를 사용해 요청을 처리합니다.

네 서비스는 제가 담당했습니다. 일기·미디어와 AI 기능은 서비스 전체 구성에 포함되지만 이 도식은 제가 구현한 인증·사용자·그룹 요청 흐름에 집중합니다.

문제 해결

로그인 후 내 정보 조회가 다시 로그인으로 돌아가던 문제

문제

Google 로그인 이후 내 정보를 조회하면 User Service가 다시 로그인 화면으로 돌려보냈습니다.

원인

Gateway와 내부 서비스가 서로 다른 인증 방식을 적용하고 있었습니다. User Service의 폼 로그인과 세션 인증이 Gateway에서 검증한 요청을 그대로 처리하지 못했습니다.

해결

각 서비스에서 인증 객체를 생성하는 방법과 Gateway에서 공통 처리하는 방법을 검토했습니다. Gateway가 쿠키의 액세스 토큰을 공개 키로 검증하고 X-User-Id 헤더로 사용자 식별자를 전달하도록 했습니다. User Service의 폼 로그인과 세션 인증을 비활성화하고, User-Group Service가 검증된 식별자를 사용하도록 인증 경계를 정리했습니다.

Google 로그인에서 사용자 정보 처리가 실행되지 않던 문제

문제

Google 로그인 후 기존 사용자 정보 처리 코드가 호출되지 않았습니다.

원인

Google은 OIDC 방식으로 동작해 일반 OAuth2 사용자 정보 처리와 실행 경로가 달랐습니다.

해결

OidcUserService 기반으로 사용자 정보 처리를 변경해 정보 저장과 로그인 성공 후 JWT 발급을 연결했습니다.

로그인 후 내부 서비스 주소로 이동하던 문제

문제

로그인 후 브라우저가 내부 주소인 auth-ms로 이동했습니다.

원인

Gateway가 Auth Service로 요청을 전달하면서 Host 헤더가 내부 서비스 주소로 바뀌었습니다.

해결

인증 경로에 PreserveHostHeader 필터를 적용해 외부 요청의 Host를 유지했습니다.

결과

Google 로그인 동작 확인

수정 후 Google 로그인 동작을 직접 확인했습니다.

인증 예외 처리 확인

Gateway 테스트에서 인증 예외 경로, 토큰 누락과 비유효 토큰 처리를 확인했습니다.

검증 범위

유효 토큰의 Gateway 통과 테스트는 비활성화된 상태였습니다. Jenkins·EC2 배포와 운영은 담당하지 않았습니다.

회고 — 서비스 규모에 맞는 선택

팀에서는 프로젝트 규모에 비해 MSA가 과하다는 의견이 있었습니다. 저는 인증 서비스와 Gateway를 직접 구현해 보는 방향을 제안했습니다. 프로젝트를 마친 뒤에는 기술적 도전과 함께 팀과 서비스 규모에 맞는 선택인지 검토해야 한다는 점을 배웠습니다.

로그인·작품·회차

로그인 프론트엔드·백엔드 / Work·Episode 백엔드 개발

[🔗 백엔드 저장소](#) [🔗 프론트엔드 저장소](#) [상세 웹페이지](#) [기](#)

프로젝트 개요

장편 창작자가 작품별 회차와 세계관을 관리하고 설정 오류 탐지 결과를 확인하는 서비스입니다. 로그인 프론트엔드·백엔드와 작품·회차 API를 개발하고 있습니다. AI 설정 오류 탐지 엔진은 제 담당 범위에 포함하지 않습니다.

주요 기능

- 작품과 회차 원고 관리
- 세계관과 설정 오류 탐지 결과 확인
- Google 로그인

담당 범위

- 로그인 화면과 반환 처리, 보호된 화면 접근 제어
- 토큰 발급·재발급, 쿠키 인증과 CSRF 처리
- 작품 생성·조회·제목 수정과 사용자별 접근 처리
- 회차 상태·원고 관리, 다운로드와 여러 회차 업로드

Spring Boot

OAuth2

JPA

아키텍처

로그인·작품·회차의 구성

담당 · 청록색 테두리와 역할 레이블

Next.js 프론트엔드

로그인 담당

로그인·작품·회차 API ↕

Spring Boot 백엔드

로그인·Work·Episode 담당

사용자·작품·회차 데이터 ↕

OAuth 요청·토큰 데이터 ↕

PostgreSQL

Redis

Next.js 프론트엔드와 Spring Boot API 서버를 연결합니다. PostgreSQL은 사용자·작품·회차 데이터를 저장하고, Redis는 OAuth 요청과 인증 토큰 데이터를 관리합니다.

현재 계층형 구조로 MVP를 먼저 구현하고 있습니다. 서비스 분리는 이후 계획이며, 지금 여러 도메인을 독립 서비스로 분리한 상태는 아닙니다.

문제 해결

브라우저와 서버의 로그인 상태 맞추기

구현 과제

로그인 반환 처리부터 보호된 화면 접근과 로그아웃까지, 브라우저 상태와 서버의 인증 처리를 함께 연결해야 했습니다.

구현

프론트엔드에서는 Google OAuth 로그인 화면과 반환 처리, 보호된 화면 접근 제어를 구현했습니다. 서버에서는 JWT 발급·재발급, Redis 기반 OAuth 요청과 리프레시 토큰 관리, 쿠키 인증과 CSRF 처리를 맡았습니다. 로그아웃 이후 브라우저 상태와 현재 사용자 정보도 함께 동기화했습니다.

작품 접근과 여러 회차 원고 처리

구현 과제

작품별 접근을 처리하고, 여러 회차의 원고 입력을 검증해 일괄 처리하는 기능이 필요했습니다.

구현

Work에서 작품 생성·조회·제목 수정과 사용자별 접근 처리를 구현했습니다. Episode에서는 회차 상태, 원고 수정·조회, UTF-8 다운로드와 여러 회차 업로드의 입력 검증·일괄 처리를 구현했습니다.

결과

로그인 흐름 구현

로그인 프론트엔드·백엔드와 로그아웃 이후의 인증 상태 동기화를 구현했습니다.

작품·회차 API와 테스트 작성

Work·Episode API와 도메인 테스트를 작성했습니다. 프로젝트는 현재 개발 중입니다.

검증 범위

여기에는 구현과 테스트 작성까지 확인된 범위를 기록했습니다. 현재 전체 테스트 통과나 운영 성과를 의미하지 않습니다.

회고 — MVP 구현을 먼저 진행

ILOG에서 서비스 규모에 맞는 기술 선택의 중요성을 배운 뒤, Lorekeeper에서는 계층형 MVP를 먼저 구현하고 이후 서비스를 분리하는 순서로 바꿨습니다. 서비스 분리는 향후 계획입니다.

팀·실시간 채팅

6인 팀 · 백엔드 팀 · 채팅 도메인

 [GitHub 저장소](#) [상세 웹페이지](#) 

프로젝트 개요

SSAFY 교육생의 프로젝트 팀 구성을 지원하는 서비스입니다. 팀 생성·조회·수정·삭제와 초대·참여 요청·거절·탈퇴, 팀 상태 잠금 기능을 개발했습니다. 팀 채팅과 1:1 채팅도 담당했습니다.

주요 기능

- 팀 탐색과 팀원 초대·참여 요청
- 팀 구성 상태 관리
- 팀 채팅과 1:1 채팅

담당 범위

- 팀 관리와 초대·참여·거절·탈퇴 처리
- 팀 상태 잠금
- 실시간 메시지 송수신과 채팅방·기록 조회 API
- 프론트엔드 채팅 연동과 팀·채팅 테스트 작성

Spring Boot

WebSocket

STOMP

아키텍처

팀·채팅 API와 실시간 메시지

담당 · 청록색 테두리와 역할 레이블

API 조회

React 화면

채팅 연동 담당

팀·채팅방·기록 조회 API ↔

Spring Boot 백엔드

팀·채팅 담당

실시간 메시지

React 화면

채팅 연동 담당

WebSocket · STOMP 메시지 ↔

Spring Boot 백엔드

팀·채팅 담당

React 화면과 Spring Boot 백엔드를 연결했습니다. 팀 관리와 채팅방·메시지 기록 조회는 API로 제공하고, 실시간 메시지는 WebSocket과 STOMP로 송수신합니다.

저는 백엔드의 팀·채팅 도메인과 프론트엔드 채팅 연동을 담당했습니다. 아래 도식은 API 조회와 실시간 메시지 전달이라는 두 흐름을 구분합니다.

문제 해결

팀 구성 단계의 여러 동작 처리

구현 과제

팀 생성 이후에도 초대·참여 요청·거절·탈퇴와 팀 상태 잠금 등 팀 구성 과정의 여러 동작을 처리해야 했습니다.

구현

팀 관리 API와 초대·참여·거절·탈퇴 처리를 구현했습니다. 팀장이 팀에서 나갈 때의 팀 처리 로직도 수정했습니다.

채팅 연결과 1:1 채팅 상태 연동

문제

프론트엔드 채팅 화면을 연결하는 과정에서 WebSocket 연결·CORS 설정 오류와 1:1 채팅 상태 처리 문제가 있었습니다.

해결

WebSocket 연결과 CORS 설정을 수정하고 1:1 채팅 상태 처리를 프론트엔드와 맞췄습니다. 팀·1:1 메시지 송수신과 채팅방 생성·메시지 기록 조회 API를 연동했습니다.

결과

팀·채팅 기능 구현과 연동

팀 관리 API, 팀·1:1 실시간 채팅과 채팅방·메시지 기록 조회를 구현했습니다. 프론트엔드 연동 과정에서 연결과 채팅 상태 처리를 수정했습니다.

단위·통합 테스트 작성

팀 서비스·컨트롤러와 채팅방·메시지·멤버 서비스의 단위 테스트 및 통합 테스트를 작성했습니다.

검증 범위

이번 기록은 구현·연동 수정과 테스트 작성 범위입니다. 현재 전체 테스트 통과나 동시 접속 성능을 측정한 결과는 포함하지 않습니다.